IMPLEMENTATION EDITION • 08 OCTOBER 2026

## Mhistree AI Agent Rebuild Playbook

# Preserve the interface. Rebuild the foundations. Deliver the care journey.

A sequential, feature-first implementation guide for an existing Mhistree project. Written for agents and developers who need small, verifiable tasks, clear ownership, and reliable handoffs.

### 36 task groups. One controlled path.

Governance and UI preservation → code audit → independent architecture → clean foundation → patient experience → practitioner experience → organization experience → integration and handover.

### The two mandatory files

AGENTS.md is the portable product brain: identity, rules, architecture philosophy, and execution discipline. WORKFLOW.md is the implementation memory: actual repository state, task progress, relevant files, evidence, dependencies, and next actions.

### What this document establishes

The accepted product decisions and the required execution process. It provides proposed paths and implementation contracts; the implementing agent must resolve them against the actual repository and selected services.

### What has not been audited

The existing source code and running interface were not provided for this document. No component, endpoint, test, backend integration, or migration is claimed to work. Preserve the real UI through inspection; do not infer its colors or layout from this PDF.

First release: the core patient-centered EHR journey. Deferred features remain in the roadmap, not in launch acceptance criteria.

OPERATING MANUAL

# How to execute this playbook

### Authority order

Latest explicit developer instruction > current AGENTS.md and WORKFLOW.md >= this playbook > existing code and older project PDFs. Code is evidence of current behavior, not authority for desired behavior. The two Markdown files have different responsibilities: AGENTS governs product invariants; WORKFLOW records execution state. Neither can silently overrule a newer developer decision.

### Resolve conflicts deliberately

Record the conflicting statements and the latest developer decision. Update both files if affected. An agent-generated edit cannot promote its own assumption into a developer decision. Ask only when a consequential ambiguity has no authorized default; otherwise apply this playbook and document the choice.

### The first two newly created project files

1. Write AGENTS.md before analyzing implementation details. 2. Conduct a read-only repository and UI inspection, keeping observations in working memory. 3. Write WORKFLOW.md from those observations. Only then create design guides, audit reports, screenshots, skills, application files, or other supporting documents. If these files already exist, preserve and reconcile them in this same order; do not overwrite developer rules blindly.

### Every task uses the same loop

Read its bounded context packet; confirm its prerequisites and the previous completed task still work; implement only the stated slice; run the specified positive and negative checks; record evidence, errors, files, and future hooks; update WORKFLOW.md before moving on. A failed gate blocks dependent work. Independent documentation may proceed if the blocked dependency is explicit.

### Keep tasks small enough to resume

Each numbered group is a cohesive capability. If it exceeds one reliable working session, split it into Txx.1, Txx.2, and so on in WORKFLOW.md. Each subtask gets inputs, outputs, tests, and a handoff. Never mark a group complete while its required subtask is blocked, mocked, or untested.

### No unnecessary full-repository rereads

After the initial audit, start from the feature index, task record, and relevant contracts. Read their listed files and direct consumers. Broaden searches only when an import, schema, failing test, or missing map provides a concrete reason. Repair the map after discovering a gap.

OPERATING MANUAL

# Product canon: identity and access

## Mhistree identity

Mhistree is a patient-centered advanced EHR with a continuing lifetime history. A person can receive care indefinitely without activating a login. A professional profile extends one account; organization membership and clinical qualification are separate. Do not import Reslz commerce concepts or design decisions.

## Person, account, and contacts

One permanent person identity is the goal. Login is optional. Email and phone are optional, separately verified contacts; neither is required for organization/practitioner registration or patient activation. A permanent Mhistree ID locates a person but grants neither authentication nor record access. Controlled search returns minimum matching information and never automatically exposes history.

## Activation and recovery

Issue a private expiring one-time code directly or a one-time email link when available. Redemption opens restricted setup so the patient sets their own password and may add contacts. Staff never set a patient password. Authorized staff verify identity before replacement of an unactivated invitation; replacement invalidates its predecessor. Activated-account recovery uses an identity-checked one-time recovery code, invalidates prior sessions, and is audited. Uncertain cases go to Mhistree support.

## Independent connection paths

A patient may connect to multiple direct practitioners and organizations, or both. Organization staff require a valid assignment and permission inside the patient consent ceiling. Every action selects an explicit direct or organization context. Revoking one path does not revoke an independent one; never silently use another path to continue an operation.

## Consent and revocation

Support login approval and identity-checked recorded in-person consent. Record categories, exclusions, timestamp, responsible staff, and method. Default clinical scope covers relevant finalized records, medications, allergies, ongoing issues, and results, with exclusions explained. It covers future finalized records in permitted categories until narrowed or revoked. Private notes and drafts are excluded. Patient revokes the organization as a whole, not individual internal assignments.

## Reconnection and paused work

Revocation immediately ends connection-derived access, including contributor access to their own entries, while preserving history. Drafts cannot be reopened, edited, or finalized. Dependent requests pause; independently authorized recipients may continue only within their own scope. Reconnection requires fresh consent and valid assignments. Authorized practitioners explicitly review, then resume, update, or cancel old work. Preserve dates and log every transition.

OPERATING MANUAL

# Product canon: records and AI

## Two layers, one coherent history

Persistent information contains attributed reports, observations, documents, clinical entries, medication use, allergies, baseline, and ongoing issues. On-demand assessment is labeled AI interpretation retained in chat; it never automatically becomes a diagnosis, timeline event, or persistent condition. A practitioner may review and finalize their own attributed conclusion.

## Natural conversation and extraction

Save clear patient-reported information automatically, linked to its source and timing. Ask about consequential ambiguity without turning chat into a questionnaire. A report of running followed by leg pain creates linked activity and symptom records without asserting causation. General questions, hypotheticals, and third-party stories are not patient facts. Unidentified medication remains a visible clarification item, not a guessed drug.

## Time and state

Store exact capture timestamps to seconds with timezone context. Event time is separate and can be exact, approximate, a range, or unknown. Anchor relative phrases to source time and timezone; do not manufacture precision. Preserve elapsed gaps within a chat. A contribution updates timeline and linked persistent state consistently. Silence does not prove healing, continued medication use, or completion; stale current status may become unknown.

## Documents, conflicts, and corrections

Save originals immediately; extracted details retain patient-uploaded source status until authorized verification. Ambiguous text, units, dates, or patient identity cannot silently update persistent state. Keep partial extraction visible and retry without duplicate events. Patients may correct their own extracted information with history preserved; they cannot rewrite finalized professional findings. Minor corrections revise the same entry; corrected results retain versions; repeated tests stay separate events. Conflicting contributions remain attributable and visible.

## AI boundaries

Use compact authorized persistent context plus relevant timeline events and originals. Source links, timing, gaps, and contradictions travel with retrieval. Duplicate accounts and old AI guesses are not independent evidence. Use insufficient/limited/moderate/strong support with explanations when appropriate; urgency is separate. AI can organize, explain, summarize, and suggest possible explanations; it does not independently confirm diagnoses, prescribe, or change prescribed treatment. Escalation language and clinical behavior require review before patient launch.

## Care and private work

Personal notes are author-only by default and stay out of patient-facing records and unrelated AI retrieval. Clinical drafts are limited to the author/permitted team. Finalized care is visible to the patient and authorized viewers. An explicit service request, not a shared tentative note, authorizes the narrow request/result workflow.

# Product canon: reconciliation and scope

## Duplicate identities

When a match is uncertain, keep a separate provisional identity so care can continue without access to the possible match. Ordinary staff can flag duplicates. Designated hospital staff verify and approve straightforward merges; uncertain identity or conflicting activated ownership goes to support. A printed ID or patient-supplied ID may help locate both records but does not prove ownership.

## Surviving identity

If one login is activated, retain it. If neither is activated, retain the oldest identity. If both are activated and ownership is verified, retain the identity with the most recent successful patient login; staff activity and record edits do not count. Resolve ties or missing login evidence through support. Retire the other login, invalidate sessions and unused tokens, and keep previous IDs as aliases. No history is deleted.

## Merge safety

Preserve original authors, institutions, timestamps, document origins, consent lineage, and versions. Log evidence, reviewer, reason, mappings, and reversal data. Merging does not widen permissions; pre-merge grants retain their scope and origin boundaries unless fresh consent explicitly changes them. A controlled reversal must account for records created after the merge; never restore old credentials or sessions automatically.

## Included in the first usable release

Patient registration and activation; approved organizations and verified practitioners; connections, consent, assignment and revocation; time-aware history and persistent health details; uploads and extraction; practitioner notes, drafts, finalized entries and corrections; patient AI chat and on-demand assessment; practitioner AI assistance; basic appointments and text messaging; narrow test requests and results; identity reconciliation; basic patient export and requests for correction/removal/closure.

## Deferred by the latest decisions

Guardianship and transfer to a child; personal-support connections; printable patient cards and access PINs; external calendar export/integration. Also outside this release: audio/video, pharmacy fulfillment, inventory, payments, broad hospital operations, full laboratory/specimen management, and mandatory full FHIR implementation. Keep hooks documented without building these systems now.

## Clinical versus operational scope

Allow authorized practitioners to record medication/prescription information and results. Patient-reported use is distinct from the prescription. Do not infer pharmacy fulfillment or electronic prescribing. Test requests are a narrow patient-linked service workflow; do not recreate hospital operations. Patient removal and account closure are requests, not automatic destruction of shared clinical history.

# Target architecture: simple ownership

Design the target from the product first. Map reusable code into it afterward. Root-level app/ is the routing layer; root-level features/ owns business behavior. This is the recommended Mhistree layout for this rebuild, not a claim about the existing repository.

| Path | Responsibility |
| --- | --- |
| app/ | Route pages, layouts, loading/error boundaries and thin endpoint adapters only. |
| features/<feature>/ | Feature UI, validation, domain rules, server operations, tests, and a concise README. |
| components/ui/; components/layout/ | Shared primitives and shells preserving the inspected UI. No patient business rules. |
| lib/ | Infrastructure clients: database, auth provider, AI gateway, private storage, jobs, logging, time. |
| database/ or provider migration folder | One authoritative migration stream, policy definitions, synthetic fixtures. Do not keep duplicate schema systems. |
| docs/; tests/ | Contracts, audit, design guide, feature map, task evidence; cross-feature integration and end-to-end tests. |
| AGENTS.md; WORKFLOW.md | Portable specification and current execution record, respectively. |

## Feature ownership

identity owns people/aliases/merges; auth owns sessions/setup/recovery; access owns consent/connections/policy; health-records owns events/state/corrections; documents owns originals/extraction jobs; ai-chat owns conversations/retrieval/assessments; patient owns patient screen composition; practitioner owns professional profile/workspace; clinical-care owns notes/entries/requests/results; appointments and messaging own their workflows; organizations owns approval/membership/assignments; audit owns accountable security records.

## Keep boundaries readable

Use plain functions and explicit types. A simple feature may need only components/, schema.ts, server.ts, and README.md. Split larger server modules into policy, service and repository files only when responsibilities justify it. No generic CRUD framework, plugin engine, or dependency-injection container merely for future flexibility. Share code after a real repeated responsibility emerges; preserve distinct domain meanings.

# Target architecture: calls and contracts

## Request path

A thin route/action validates the request and session, then calls a feature server operation. That operation checks current authorization, applies domain rules, and performs a transaction through a repository or database adapter. Return a minimal display model. UI does not directly mutate clinical tables. Server Components handle suitable reads; Client Components cover interaction. Never rely on a hidden button or route redirect as data authorization. [R1-R3]

## AccessContext contract

Every protected operation receives a server-established actorAccountId, patientPersonId, actingContext (patient/direct/organization), connectionId where required, and organizationId/assignmentId when applicable. Never trust client-supplied role or consent assertions. Evaluate current verified status, active connection, category scope/exclusions, assignment, privilege, resource provenance and session. Deny by default; audit consequential decisions.

## Contribution contract

A health contribution carries sourceId/sourceVersion, patientPersonId, author identity, original organization/context, kind, body-site/context links, reportedAt, occurredAt or interval, timePrecision, sourceTimezone, sourceStatus, lifecycle status and supersedesId where applicable. Preserve raw source. Accepting a contribution writes the event and current-state change atomically; source-based idempotency prevents duplicate writes on retry.

## Future hooks that exist before future screens

Patient work uses real AccessContext and contribution contracts from the start. Practitioner screens add direct-provider operations using those same contracts. Organization screens supply organization context and assignments, not a second clinical engine. Store provenance and grant lineage before implementing merges. Task records name the later consumer and its gate.

## Backend and authentication decision

The saved recommendation favors one Postgres/Supabase backend with private files and selected embeddings, and reuse of a valid AI gateway. Treat this as a candidate to verify, not a reason to keep incompatible code. Supabase password sign-in is documented around email/phone; contactless login therefore needs an explicit provider compatibility spike. [R4] Prefer a supported username/opaque-login authentication adapter if necessary. Do not invent patient contact details, expose service credentials, or hand-roll cryptography to make the requirement fit.

## Implementation defaults, not extra product promises

Use TypeScript and a modular Next.js App Router application. Preserve the existing styling primitives when suitable. Pin verified dependency versions and commit one lockfile. Use durable background work for extraction where needed, with bounded retries and visible status. Keep production data, migration policy and auth integration separate from the choice to rebuild source code.

# Navigation and execution map

| Group | Tasks / pages | Exit condition |
|---|---|---|
| Preservation and planning | T01-T05 / 9-13 | Portable rules, inspected design, audited reuse plan, target contracts. |
| Shared foundations | T06-T09 / 14-17 | Clean shell, identity/auth/access and atomic record core verified. |
| Patient first | T10-T18 / 18-26 | Real login, chat, uploads, health views, consent and care screens work. |
| Practitioner next | T19-T25 / 27-33 | Verified direct care: connection, notes, entries, requests and workspace. |
| Organizations last | T26-T31 / 34-39 | Approval, assignments, registration, recovery and merges work. |
| Release and handover | T32-T36 / 40-44 | Cross-context gates, operations, migration, acceptance and docs verified. |

### Patient phase has a real boundary

By T18 a patient can register/activate, log in, chat with the AI, upload and process a document, view timeline and persistent health state, correct patient-reported extraction, and manage account/consent screens. Provider-facing interactions are exercised through real services and synthetic fixtures until their screens arrive. A fixture-backed provider test is evidence of a contract, not proof of a finished practitioner workflow.

### Do not delay security until organizations

Identity, access policy, audit, record provenance, and organization-capable context contracts are foundations. Building them early does not mean building organization screens early. This prevents patient and practitioner code from being rewritten when organization workflows are introduced.

### Dependency rule

Execute T01-T36 in order by default. A task needs its direct contracts as well as the preceding task gate. Before starting, rerun the preceding task smoke check and the relevant contract tests. Record the actual commands from package scripts; do not assume a script exists or invent a passing result.

### Task page vocabulary

Read = bounded context. Change = intended outputs, to be resolved to real paths in WORKFLOW. Steps = ordered implementation. Gate = observable acceptance, including misuse/failure behavior. Handoff = facts the next agent must carry forward. Every task also updates its workflow row and relevant feature README.

TASK GROUP • EXECUTE SEQUENTIALLY

# T01 Write the brain and freeze design intent

### Read before starting

Latest developer instructions and this playbook only. Do not inspect implementation internals before drafting the portable product rules.

### Create or change

AGENTS.md first; no other new project file yet.

### Step-by-step

1. Write a self-contained AGENTS.md using the contract in Appendix A. Include all product canon pages, scope boundaries, authority order, architecture principles, and execution rules; do not reduce them to links to this PDF.

2. State the UI preservation rule: inspect actual components, spacing, type, colors, motion and responsive behavior before changing them. This PDF supplies no replacement brand palette.

3. Write dos: small coherent changes, source attribution, server authorization, explicit errors, meaningful names, targeted tests, documentation at each gate. Write don’ts: redesign, silent scope expansion, guessed clinical facts, secrets in logs, destructive rebuilds, unexplained abstractions.

4. Mark the design measurements as awaiting read-only inspection. Explain that exact visual reconstruction in an empty folder requires the later verified design annex and assets. Never invent measurements to make the file look complete.

5. If an AGENTS.md already exists, reconcile its legitimate rules and preserve developer changes. Record unresolved contradictions for the audit without copying legacy product behavior into the canon.

### Gate: prove this before continuing

The file stands alone for product identity, permissions, AI boundaries, scope and build order; contains no unverified claim that code works; is the first new/updated governing file. No app or styling changes.

### Handoff and future dependency

T02 reads this file before the repository. T03 adds observed visual rules, clearly separated from implementation findings. A portable text spec cannot reproduce absent proprietary assets byte-for-byte.

TASK GROUP • EXECUTE SEQUENTIALLY

# T02 Audit read-only; create execution memory

### Read before starting

AGENTS.md; repository inventory and existing package/config files, route entrypoints, schema, auth, AI, tests and UI. Read old PDFs only as legacy evidence.

### Create or change

WORKFLOW.md as the second new project file. Supporting audit files may be created only after it exists.

### Step-by-step

1. Inspect git state and find the actual app root. Identify developer changes; do not reset, stash or delete them. List files before broad searches, then trace representative UI and data flows.

2. Read package scripts and dependency versions. Inspect identity, contacts, roles, access, uploads, extraction, records, provider and organization code; label each capability working, partial, mocked, absent, or unverified.

3. Run available safe baseline checks and inspect the running UI if possible without changing source. Record command, result and environment blockers. Missing credentials are a blocker, not evidence of a broken design.

4. Create WORKFLOW.md with current-state evidence, all T01-T36 rows, dependencies, exact relevant files where known, target paths, baseline failures, next task and deferred hooks. Never claim a target path already exists.

5. Now create docs/audit/current-state.md and docs/feature-map.md; move detailed evidence there and keep a concise indexed workflow. Classify keep/adapt/replace/remove/defer, with reason and owning task for each area.

### Gate: prove this before continuing

The two-file order is satisfied; each reuse decision has a code path and evidence or is explicitly unverified. Existing tests and mocks are distinguished. No legacy removal or schema mutation occurred.

### Handoff and future dependency

T03 consumes the observed components. T04 consumes the audit only to constrain migration/reuse, not to dictate the target domain architecture.

TASK GROUP • EXECUTE SEQUENTIALLY

# T03 Capture and preserve the UI system

### Read before starting

AGENTS.md; WORKFLOW.md; UI audit; actual global styles, shared primitives, shells and representative patient/provider pages.

### Create or change

docs/design/UI_GUIDE.md; docs/design/component-catalog.md; sanitized reference captures; a measured design annex in AGENTS.md.

### Step-by-step

1. Inspect rendered desktop and mobile screens. Record actual tokens, fonts/assets, spacing rhythm, radii, borders, shadows, navigation, density, typography, interactions and responsive transitions.

2. Catalog each reusable component: source path, purpose, props, variants, states, layout rules, accessibility, examples and known defects. Distinguish visual pattern from outdated data behavior.

3. Write a Markdown guide future agents must read before UI work. If adding executable agent skills, follow the repository's skill format; plain design guides are sufficient when no skill system exists.

4. Capture baseline screens without real patient information. Define comparison viewports and key flows. Do not redesign to match this PDF. Where the running app is unavailable, record the missing visual verification and preserve source assets.

5. Add verified design tokens and essential layout rules to AGENTS.md for portability. Index external assets and license/source needs; keep code-path-specific notes in the catalog.

### Gate: prove this before continuing

A developer can reproduce one representative component and page from the guide and reference assets. Compare desktop/mobile and focus/error/empty states. Every uncertain design property is marked unresolved.

### Handoff and future dependency

All later UI tasks read this guide and cite the components reused. Any necessary accessibility repair preserves visual intent and records its reason; visual drift blocks the task gate.

TASK GROUP • EXECUTE SEQUENTIALLY

# T04 Design an independent feature-first target

### Read before starting

Product canon; design guide; audit summary; official documentation for the selected framework and services.

### Create or change

docs/architecture.md; docs/contracts/identity-access.md; docs/contracts/clinical-records.md; docs/decisions/stack.md.

### Step-by-step

1. Model the product from its accepted workflows without copying legacy folders or tables. Use the proposed ownership map; resolve overlaps to one owner each.

2. Specify route groups and feature APIs. Keep business components out of app/ route folders. Write import rules, server/client separation and minimal data-transfer shapes.

3. Define AccessContext, consent lineage, contribution, correction, merge and job contracts. Specify transactions, idempotency, error codes, pagination and version conflict behavior.

4. Verify contactless activation/login support with the auth provider before committing to it. Document a supported identifier-based adapter if the existing provider cannot serve this flow; real patient email/phone remain optional.

5. Map audit candidates to target modules after architecture is complete. Record which infrastructure can remain, which source modules can be adapted, and migration boundaries. Avoid extra services unless a concrete requirement needs them.

### Gate: prove this before continuing

A trace from patient chat to saved event and a trace from organization staff to authorized record read have explicit owners and no circular feature dependency. Contactless auth has a documented feasible route and test plan.

### Handoff and future dependency

T05 turns contracts into dependency/file maps. Provider names and package choices remain replaceable infrastructure details; product rules remain fixed unless the developer changes them.

TASK GROUP • EXECUTE SEQUENTIALLY

# T05 Plan the rebuild and verification gates

### Read before starting

Architecture/contracts; baseline failures; reuse matrix; WORKFLOW task rows.

### Create or change

docs/migration-plan.md; docs/testing.md; updated WORKFLOW and feature map.

### Step-by-step

1. Choose an isolated branch/worktree or new application directory; preserve the old source and data. Define which area is the new app and how it runs independently.

2. Expand tasks into bounded subtasks where needed. Each lists exact files to read/create/change, prerequisites, tests, expected output, errors, and future consumers.

3. Define synthetic fixtures for two patients, two practitioners, two organizations, revoked/direct/org access, incomplete extraction and duplicates. No real clinical data in tests.

4. Separate source rebuild, database migration, auth migration and production cutover. Plan backups, reconciliation, rollback, and safe reuse imports. Do not delete the old app to make room.

5. Record baseline commands and measurable gates. Use one acceptance ledger to distinguish implemented, verified, blocked, deferred and mock-only items.

### Gate: prove this before continuing

Every T06-T36 capability has an owner and gate; future practitioner/org dependencies are explicit; the old application remains recoverable; no build step depends on an undocumented stub.

### Handoff and future dependency

T06 starts the new codebase. Stack constraints or unavailable credentials should be recorded as blockers with a concrete next action, not hidden behind successful mock tests.

TASK GROUP • EXECUTE SEQUENTIALLY

# T06 Create the clean application foundation

### Read before starting

Target architecture; design guide; reuse matrix; migration plan; package choices.

### Create or change

New app/, features/, components/, lib/; package scripts and lockfile; environment example; base test setup.

### Step-by-step

1. Initialize the selected pinned Next.js/TypeScript setup in the planned location. Use root-level app/ and features/ by default; no duplicated source trees.

2. Restore inspected fonts, tokens, shell, navigation and shared UI through reviewed reuse. Remove outdated feature behavior from copied visual components.

3. Add typed environment validation with example variable names only. Place secrets and privileged clients behind server-only boundaries.

4. Implement loading, empty, unauthorized, error and retry patterns matching the existing UI. Make core routes render without pretending unfinished features work.

5. Add lint/type/build scripts and targeted test runners. Record run instructions for an intermediate developer.

### Gate: prove this before continuing

Fresh dependency install and build succeed; representative mobile/desktop screens match the baseline; client bundles contain no server secret; error boundaries work. Unbuilt capabilities are visibly unavailable.

### Handoff and future dependency

T07 adds durable identity. Shared components remain presentation-focused; later features compose them rather than forking visually similar copies.

TASK GROUP • EXECUTE SEQUENTIALLY

# T07 Separate people, accounts and identifiers

### Read before starting

Identity contract; chosen auth compatibility decision; database adapter and migration conventions.

### Create or change

features/identity/; foundational schema/migrations; synthetic identity fixtures and tests.

### Step-by-step

1. Create Person, Account link, optional Contact with verification, PatientIdentifier aliases, and provisional/matched status. Keep auth provider subject separate from the permanent person key.

2. Permit person creation with basic known details and no phone/email. Preserve unknown values rather than fabricating dates or contact information.

3. Implement controlled exact-ID lookup and limited candidate search with minimum demographic output, rate limits and review flags. Lookup must not return clinical history.

4. Create registration operations usable later by an authorized practitioner or organization. Capture creator, context and original record ownership.

5. Reserve merge lineage and alias uniqueness contracts; do not implement destructive merge shortcuts.

### Gate: prove this before continuing

A person without an Account or contacts can be stored with creator/context attribution and referenced by a synthetic care fixture. Similar names/shared phone do not auto-link. Guessing an ID does not expose history.

### Handoff and future dependency

T08 attaches login; T20/T28 call the same registration operation. T31 relies on stable origin IDs and aliases, so these fields must not be discarded.

TASK GROUP • EXECUTE SEQUENTIALLY

# T08 Implement sessions, activation and recovery core

### Read before starting

Auth adapter proof; identity contract; token/session policies; current provider docs.

### Create or change

features/auth/; lib/auth/; token/session persistence; setup/recovery tests; docs/contracts/auth.md.

### Step-by-step

1. Implement login with a supported opaque identifier/username plus password route for patients without contacts, alongside applicable verified-contact flows. Do not synthesize personal contact information.

2. Create cryptographically random, hashed-at-rest, expiring single-use setup/recovery credentials. Redeem atomically; restrict setup sessions until the patient sets their password.

3. Add replacement/recovery service operations guarded by verified staff privileges and identity-check evidence. Staff-facing screens arrive later. Separate unactivated invitation replacement from activated recovery.

4. Invalidate replaced tokens and prior sessions when required; enforce session freshness server-side. Add attempt limits, non-enumerating errors, logout and safe reset behavior.

5. Test auth-provider boundaries with real integration behavior in the test environment; record any provider limitation blocking the contactless path.

### Gate: prove this before continuing

No-contact patient activates and logs in; reused/expired/replaced code fails; concurrent redemption succeeds once; recovery retires old sessions; staff cannot choose a patient password or impersonate them.

### Handoff and future dependency

T10 uses real login/setup screens. T29 exposes staff recovery services. If contactless auth cannot pass, block dependent completion rather than silently requiring email.

TASK GROUP • EXECUTE SEQUENTIALLY

# T09 Build authorization, audit and record kernel

### Read before starting

AccessContext and contribution contracts; identity/auth APIs; role and privilege definitions.

### Create or change

features/access/, features/audit/, health-records server kernel; policy tests and migrations.

### Step-by-step

1. Implement current-state policy evaluation for patient/direct/organization contexts. Add consent categories/exclusions, active/revoked status, assignments, verification and resource provenance checks.

2. Implement access-enforced event/current-state writes in one transaction and source-version idempotency. Create immutable origin metadata and controlled correction history.

3. Record meaningful access, consent, activation/recovery and clinical mutations without raw secrets or unnecessary clinical payloads in operational logs.

4. Protect data and private file access at the server and database layer where exposed. Ensure background jobs use narrowly scoped authority and recheck before publication.

5. Create negative fixtures now for cross-patient, cross-organization, revoked, draft and private-note access. Implement synthetic provider grants through test setup only.

### Gate: prove this before continuing

Tampered IDs and roles fail; revocation changes the next authorized operation; no partial event/state update on failure; replay writes no duplicate; independent paths remain isolated.

### Handoff and future dependency

All later reads, mutations, AI retrieval and jobs must call this kernel. Organization UI is future work, but its policy semantics are already real and tested.

TASK GROUP • EXECUTE SEQUENTIALLY

# T10 Deliver patient registration and account entry

### Read before starting

Auth/identity APIs; UI guide; access rules; account setup routes.

### Create or change

features/patient/account/ and auth components; thin signup/login/setup routes.

### Step-by-step

1. Build patient self-registration, code/link redemption, password setup, login, logout and profile/contact editing using the real auth core.

2. Support hospital-created identities without re-registering them. An invitation claims the existing Person rather than generating another patient record.

3. Keep phone/email optional and show verification state separately. Provide identifier-based login guidance without making public ID alone an authenticator.

4. Use clear pending/expired/replaced/invalid invitation states and recovery directions. Prevent contact edits from silently transferring account ownership.

5. Verify keyboard/mobile behavior and duplicate-submit handling; update the patient feature map.

### Gate: prove this before continuing

Patient signs up or claims an existing identity, refreshes the session, signs out and returns. Missing contact never blocks setup. Protected pages reject unauthenticated access directly.

### Handoff and future dependency

T11 assumes a durable patient session and Person mapping. Assisted staff flows remain scheduled for T29, with their server operations already present.

TASK GROUP • EXECUTE SEQUENTIALLY

# T11 Build patient home and durable AI chat

### Read before starting

Patient/auth map; message/source/time contract; AI gateway boundary; design guide.

### Create or change

features/patient/home/; features/ai-chat/ conversation persistence and UI; thin chat route.

### Step-by-step

1. Create patient home with truthful empty/loading states and entry points for chat, documents, health details and connections.

2. Persist patient messages with exact capture time, timezone and stable IDs before dependent extraction. Persist AI replies separately from patient-source evidence.

3. Integrate the AI gateway server-side with bounded context, cancellation/retry behavior and visible errors. If generation fails, retain the patient message.

4. Keep conversation history distinct from health events. Preserve message ordering and elapsed gaps across morning/evening or resumed sessions.

5. Use idempotent send requests to prevent duplicate messages on retry and reconnect. Do not stream unauthorized context into the browser.

### Gate: prove this before continuing

Patient chats, reloads and sees durable history; failed generation is recoverable; a double click creates one message; another patient cannot fetch the conversation; no AI reply becomes a clinical fact.

### Handoff and future dependency

T12 processes the persisted patient source. T16 enriches retrieval; basic chat must be honest about unavailable assessment context until that gate passes.

TASK GROUP • EXECUTE SEQUENTIALLY

# T12 Extract reports with temporal context

### Read before starting

Contribution contract; persistent chat sources; extraction schemas; health-record kernel.

### Create or change

features/ai-chat/extraction/ or one documents/shared extraction owner; temporal parsers; contribution tests.

### Step-by-step

1. Define validated extraction output for symptom, activity, medication report, allergy/baseline and status change, preserving source spans and context links.

2. Capture clear patient reports automatically; exclude general questions, hypotheticals and third-party content. Route consequential ambiguity to clarification rather than guessed fields.

3. Resolve relative dates from source timestamp/timezone; retain approximate intervals or unknown time. Keep capture time distinct from event time.

4. Link activity and symptom without claiming causation. Preserve unresolved medication reports for later clarification without inserting a guessed drug into identified medication state.

5. Write accepted contributions through the atomic kernel; use source/version/item idempotency and visible partial/failed status.

### Gate: prove this before continuing

Fixtures cover running plus pain, hypothetical pain, “other leg,” unknown medicine, “yesterday” across midnight and timezone, approximate month, partial failure and replay. No invented exact time, diagnosis or duplicate.

### Handoff and future dependency

T13 uses the same source/provenance model for documents. T14/T15 consume accepted events and current-state projections, not raw unvalidated model output.

TASK GROUP • EXECUTE SEQUENTIALLY

# T13 Add private documents and resilient extraction

### Read before starting

Document contract; storage adapter; extraction kernel; patient upload UI pattern.

### Create or change

features/documents/; private file routes; extraction job worker/status UI; source references.

### Step-by-step

1. Implement authenticated upload with patient ownership, size/type limits, server-side validation and a quarantine/scan path appropriate to supported files. Persist the original and checksum before extraction.

2. Create a durable queued job with attempts, timestamps, partial results and safe retry. Retain original content when parsing fails; display incomplete processing.

3. Extract readable details with page/span references and patient-uploaded source status. Verify patient identity relevance before applying to that patient; quarantine mismatches for clarification.

4. Require clarification for consequential unclear units, values, names or dates. Prevent text inside documents from instructing the AI to bypass policies or call tools.

5. Authorize each preview/download. Prefer controlled delivery that rechecks current access; document unavoidable limits for already downloaded copies.

### Gate: prove this before continuing

Upload survives parser outage; retry creates no duplicate event; wrong-patient/unclear-unit document does not contaminate state; unauthorized preview fails; original is recoverable and linked to extracted items.

### Handoff and future dependency

T14 shows documents beside events. T23 later adds provider-verified result workflows; uploading alone must never mark findings clinically confirmed.

TASK GROUP • EXECUTE SEQUENTIALLY

# T14 Render the source-linked patient timeline

### Read before starting

Health-record contribution API; document references; time model; timeline design patterns.

### Create or change

features/health-records/timeline/; patient timeline page; query/pagination tests.

### Step-by-step

1. Build a paginated chronological view of accepted health events, distinct from chat history. Provide category and source filtering without fetching the entire lifetime at once.

2. Show event time precision and submission time separately. Place unknown-date events in an explicit area/order; do not silently order them as newly occurred.

3. Expose attribution and original source navigation: message or permitted document page. Display linked activity/symptom context and corrections.

4. Show latest corrected version by default with accessible version history. Repeated tests remain distinct; superseded corrections are labeled.

5. Apply current permission checks to source previews as well as timeline items. Preserve responsive readability and empty/error states.

### Gate: prove this before continuing

Exact, approximate and unknown dates are distinguishable; source links point to the correct material; pagination does not skip/duplicate events; revoked source previews cannot leak content.

### Handoff and future dependency

T15 adds persistent state beside the timeline. Provider screens reuse this component with authorized projections, not a forked record interpretation.

TASK GROUP • EXECUTE SEQUENTIALLY

# T15 Build persistent health details and corrections

### Read before starting

Record kernel; temporal/status rules; patient correction permissions; timeline APIs.

### Create or change

features/health-records/current-state/ and corrections; patient health summary; status/correction tests.

### Step-by-step

1. Render medications, allergies, baseline and ongoing issues with source, status, last-known time and uncertainty. Persistent placement does not imply greater truth.

2. Implement medication use states and separate planned course from actual use. Patient stopping updates their report without rewriting the practitioner prescription.

3. Track ongoing/resolved/recurrent/unknown status explicitly; stale silence cannot silently close a condition or confirm continuing use. Ask relevant status questions naturally.

4. Allow patient correction of their reported/extracted details with original source and attributed version history. Flag finalized professional entries for professional correction.

5. Update timeline-linked state consistently under a transaction and optimistic version check; surface conflicts rather than overwriting concurrent edits.

### Gate: prove this before continuing

Left/right correction updates the linked display without erasing source; professional finding cannot be edited by patient; stale medication course is uncertain; concurrent edits and rollback preserve consistency.

### Handoff and future dependency

T16 retrieves these compact attributed states. T22 adds professional correction operations using the same version model, not a new history store.

TASK GROUP • EXECUTE SEQUENTIALLY

# T16 Implement retrieval and on-demand assessment

### Read before starting

AI behavior canon; current-state/time/source APIs; access policy; retrieval index design.

### Create or change

features/ai-chat/retrieval/ and assessment; prompt/evaluation fixtures; index refresh jobs.

### Step-by-step

1. Build context from authorized compact persistent state, time-aware conversation and relevant timeline/originals. Combine exact filters and semantic retrieval only where useful.

2. Preserve source links, activity-symptom context, event age, uncertainty, correction status and conflicting findings. Keep summaries/indexes derived and rebuildable.

3. Generate requested assessments or responses to material new evidence with labeled possibilities, support, gaps and separate urgency. Do not count repeated reports or prior AI output as independent evidence.

4. Keep assessment output in chat; do not auto-write diagnoses or treatment changes. Avoid repeatedly asking irrelevant clarification questions.

5. Filter access before retrieval and recheck before delivery; isolate patient/provider/private-note contexts. Invalidate affected indexes/caches when source or permission changes.

### Gate: prove this before continuing

Grounded synthetic cases cite their actual sources; insufficient evidence abstains; old symptoms are not assumed current; a revoked source and private note never appear in patient responses; failed extraction is disclosed as a gap.

### Handoff and future dependency

T24 reuses this engine for practitioner review with distinct allowed inputs. Clinical escalation behavior and prompts remain a named review item at release, not an agent-certified medical policy.

TASK GROUP • EXECUTE SEQUENTIALLY

# T17 Expose patient consent and account controls

### Read before starting

Connection/consent APIs; scope categories; record correction/export policy; auth privacy controls.

### Create or change

features/access/patient-controls/; patient settings; access history and export/request services.

### Step-by-step

1. Show direct practitioner and organization connections separately, including scope, exclusions, consent method and meaningful access history.

2. Implement approval, narrowing and immediate revocation for each connection. Explain that independent connections remain and previously downloaded copies cannot be recalled.

3. Require fresh consent on reconnection, with prior scope as a starting point. Never automatically resume paused drafts/requests.

4. Provide patient record export through authenticated, scoped delivery; removal/account-closure requests enter a review queue. Do not hard-delete clinical history based on a button click.

5. Use synthetic pending connection fixtures until provider invitation screens exist. Mark producer UI dependencies explicitly in WORKFLOW.

### Gate: prove this before continuing

Revoking organization A leaves direct practitioner B unaffected; narrowed categories vanish from subsequent provider reads; unactivated in-person consent can later be reviewed; export is private and closure is a request.

### Handoff and future dependency

T20 supplies direct connection producers, T27 organization producers. T32 verifies revocation across every consumer and cached view.

TASK GROUP • EXECUTE SEQUENTIALLY

# T18 Complete the patient phase and care surfaces

## Read before starting

Patient feature map T10-T17; appointment/message/request contracts; UI guide.

## Create or change

Patient appointment, text inbox and request/result views; phase acceptance evidence.

## Step-by-step

1. Implement shared appointment state services: request, accept/decline, reschedule and cancel, with timezone and overlap protection. Build patient screens first.

2. Implement patient-to-provider text thread services and patient inbox using explicit participant authorization. Separate care messaging from patient AI chat.

3. Build request/result status views consuming the clinical-care contract. Use synthetic service data until provider request screens arrive; do not advertise live fulfillment.

4. Verify the full patient journey: registration/login → chat → extraction → upload → timeline/state → correction → assessment → consent controls.

5. Record which provider interactions are contract-tested but await producer screens. Freeze stable interfaces before practitioner development.

## Gate: prove this before continuing

Core patient journey works end-to-end with real persistence and AI integration or clearly recorded external blockers. Patient messages/booking transactions are durable; unavailable provider actions are labeled; UI matches references.

## Handoff and future dependency

T19 starts practitioner work only after this gate. T23/T25 replace synthetic producers through shared services; T35 must rerun the journey with real provider and organization screens.

TASK GROUP • EXECUTE SEQUENTIALLY

# T19 Add practitioner verification and workspace access

### Read before starting

Account/profile model; clinical privileges; staff verification policy; patient-phase gate.

### Create or change

features/practitioner/profile/ and verification; minimal support review UI; practitioner layout.

### Step-by-step

1. Extend the existing account with a professional profile; do not force a second patient/practitioner login identity.

2. Support credential upload and manual review against an authoritative source, with pending/approved/rejected/suspended states and reviewer evidence.

3. Separate qualification from organization sponsorship or job title. Credential-dependent actions remain blocked without confirmed qualification and explicit privilege.

4. Build practitioner login routing and workspace shell using existing auth and UI patterns. Protect direct URLs and service calls, not just menus.

5. Provide minimal restricted review tooling needed to operate verification now; broader organization approval follows in T26.

### Gate: prove this before continuing

Pending or suspended practitioner cannot perform clinical privileged actions; ordinary user cannot self-assign verified status; approved practitioner enters the workspace with one account.

### Handoff and future dependency

T20 uses verified practitioner identity for direct patient relationships. T27 adds organization assignments without replacing the professional profile.

TASK GROUP • EXECUTE SEQUENTIALLY

# T20 Connect direct practitioners to patients

### Read before starting

Identity lookup; consent APIs; practitioner verification; in-person consent contract.

### Create or change

features/practitioner/patients/; direct connection initiation/consent screens; registration adapter.

### Step-by-step

1. Build My patients for direct connections, with controlled lookup and permission-aware search. Show no historical care merely because an identity is found.

2. Initiate patient approval or identity-checked recorded in-person consent, including categories/exclusions, method, time and responsible verified actor.

3. Allow authorized practitioner registration of an unactivated person without contacts; reuse identity/activation operations with the required privilege.

4. Implement revoke/reconnect behavior through the shared access service. Patient controls from T17 reflect changes immediately.

5. Reserve an explicit organization context selector contract; do not display unimplemented organization capabilities as working.

### Gate: prove this before continuing

Existing patient approves direct connection and permitted history appears; no consent means no cross-provider history; in-person consent works without a login; unverified practitioner cannot create privileged grants.

### Handoff and future dependency

T21 renders authorized patient workspaces. T27 adds organization-assigned patients to the same list with visible context and no accidental permission union.

TASK GROUP • EXECUTE SEQUENTIALLY

# T21 Build practitioner record views and private work

### Read before starting

Timeline/current-state components; direct access policy; private-note/draft distinction.

### Create or change

Practitioner patient record screen; clinical-care private notes, reminders and draft shell.

### Step-by-step

1. Compose permitted allergies, medication use, ongoing issues and baseline first, then recent history, documents and outstanding work.

2. Create author-only private notes and reminders separate from clinical entries and patient timeline. Model clinical drafts separately with explicit permitted-team access.

3. Show active acting context and source attribution on patient pages. Use minimal authorized projections rather than fetching hidden records to filter in the browser.

4. On revocation, clear displayed sensitive state and block new note/draft reads or writes through that path. Historical storage remains intact.

5. Document which note inputs may be used by practitioner AI and which must never reach patient-facing retrieval.

### Gate: prove this before continuing

Patient and other practitioners cannot read private notes; permitted team sees only allowed drafts; revoked author cannot reopen drafts through the revoked connection; organization context cannot be impersonated.

### Handoff and future dependency

T22 finalizes clinical entries. T24 can read private notes only in the authorized author context; publication requires a deliberate reviewed clinical contribution.

TASK GROUP • EXECUTE SEQUENTIALLY

# T22 Finalize clinical care and preserve corrections

### Read before starting

Clinical entry/version contract; draft access; practitioner privileges; atomic record kernel.

### Create or change

features/clinical-care/entries/; findings/medication record forms; finalize/correct operations.

### Step-by-step

1. Build draft care entries for consultation findings, assessments and permitted medication/prescription records. Preserve suspected diagnoses as suspected.

2. Validate current authority and version before finalization. Commit finalized entry, timeline contribution and affected persistent state atomically.

3. Make finalized care visible to the patient and authorized viewers; do not allow organizations to hide finalized clinical care.

4. Implement minor correction on the same entry with a change log and material clinical correction through a retained version/supersession path. Patient disagreement creates a review request.

5. Treat revocation as a hard pause before finalization; restored access requires explicit review/resume. Keep previous content, authors and timestamps.

### Gate: prove this before continuing

Finalized entry appears in patient view; replay does not duplicate; revoked finalize fails; patient cannot overwrite professional finding; concurrent correction requires conflict resolution and preserves every version.

### Handoff and future dependency

T23 reuses versioning for results. T28 organization contributors use the same finalization service with assignment/privilege context.

TASK GROUP • EXECUTE SEQUENTIALLY

# T23 Implement narrow test requests and results

### Read before starting

Service request contract; access policy; document and clinical version APIs; patient request views.

### Create or change

features/clinical-care/requests/ and results; authorized recipient workflow; status transitions.

### Step-by-step

1. Create explicit requests with patient, named test/service, requester, recipient, priority, dates and authorization basis. Private notes never become orders automatically.

2. Implement requested, accepted/in-progress, performed, result-draft, finalized, cancelled and paused states with allowed transitions and audit.

3. Verify recipient authorization separately; share only needed request information. Result submission/finalization requires explicit privilege, not any connected account.

4. Offer New test result versus Correct existing result. Retain original document/version; serial tests remain separate clinical events.

5. Pause when required authorization is revoked. Reconnection restores eligibility, not automatic execution; authorized practitioner reviews/resumes. Allow continuing work only under an explicit independent valid basis.

### Gate: prove this before continuing

Patient sees request status and finalized result; unauthorized recipient cannot act; result correction preserves history; new repeat test stays separate; revoked publication fails and resume requires review.

### Handoff and future dependency

T28 adds organization departments/assignees through these same services. Do not add specimen, inventory, billing or laboratory operations as implicit dependencies.

TASK GROUP • EXECUTE SEQUENTIALLY

# T24 Add practitioner AI assistance with review

### Read before starting

Retrieval/assessment engine; note/draft isolation; practitioner access and finalization APIs.

### Create or change

Practitioner AI summary, record Q&A and note-draft UI; role/context evaluation fixtures.

### Step-by-step

1. Reuse authorized retrieval for patient summaries, questions and on-demand assessment. Include source references, gaps, dates and contradictory evidence.

2. Permit author private-note context only in a clearly bounded author workspace; never leak it to patient chat, external practitioners or unrelated organization staff.

3. Generate editable draft suggestions with visible AI origin. Require practitioner review before saving an attributed clinical entry.

4. Keep unsupported hypotheses out of persistent facts. Professional finalization records the human author and any retained AI-assistance provenance.

5. Recheck access before presenting long-running generation results. Stop or withhold delivery if scope changes.

### Gate: prove this before continuing

AI can summarize authorized records with sources; injected document instructions do not bypass policy; private notes stay isolated; no AI suggestion finalizes itself; revoked in-flight response is not delivered.

### Handoff and future dependency

T25 completes practitioner daily work. T32 tests AI access isolation across direct and organization contexts and after merges.

TASK GROUP • EXECUTE SEQUENTIALLY

# T25 Complete practitioner daily work

### Read before starting

Appointment/message services from T18; practitioner patient/workspace map; request/result events.

### Create or change

Today dashboard, availability/calendar, provider text inbox, reminders and direct-practitioner gate evidence.

### Step-by-step

1. Build Today with appointments, unread messages, outstanding requests, new results and personal reminders, all permission-filtered.

2. Add availability and provider booking accept/decline/reschedule/cancel screens. Enforce concurrent slot decisions server-side; no external calendar integration in this release.

3. Build text consultations on shared messaging services with participant checks and visible delivery/failure state. No audio/video UI.

4. Wire notifications to durable clinical events with deduplication and minimal sensitive content. Do not interpret an unread message as a completed clinical task.

5. Run a real patient-to-direct-practitioner journey from connection through appointment, text, entry, test request/result and patient review.

### Gate: prove this before continuing

Direct care journey passes; double booking and unauthorized thread access fail; revoked connection blocks new work; notification retry does not duplicate; UI remains consistent with the original design.

### Handoff and future dependency

T26 begins organization work only after direct care is verified. Organization extension changes context and assignment, not patient record semantics.

TASK GROUP • EXECUTE SEQUENTIALLY

# T26 Register and approve organizations

### Read before starting

Organization contract; admin privilege model; verification tooling; practitioner phase evidence.

### Create or change

features/organizations/registration/ and approval; organization status/membership schema; admin review UI.

### Step-by-step

1. Implement organization self-registration with manual Mhistree admin approval, preserving submitted information and review evidence.

2. Model pending, approved, rejected and suspended states. Verify the initial authorized organization administrator before granting management access.

3. Separate organization administration from clinical qualification and record access. Admin status alone is not permission to browse all patient histories.

4. Create the organization shell and status screens from existing design primitives. Block pending/suspended organizations at service boundaries.

5. Add approval/suspension audit and notification handling with no unnecessary patient data.

### Gate: prove this before continuing

Pending organization cannot deliver organizational clinical actions; unauthorized user cannot approve an organization or take ownership; approved admin can manage only their organization.

### Handoff and future dependency

T27 introduces staff membership and assignments. Existing direct-practitioner access remains independent from organization status.

TASK GROUP • EXECUTE SEQUENTIALLY

# T27 Add staff roles, assignments and context switching

### Read before starting

AccessContext; organization approval; practitioner verification; direct My patients UI.

### Create or change

Organization staff/role/assignment modules; scoped consent producers; shared patient list/context selector.

### Step-by-step

1. Add staff invitation/membership and explicit operational/clinical privileges. Organization affiliation never substitutes for credential verification.

2. Connect organization to patient with login or recorded in-person consent. Organization assignments must stay within the consent ceiling.

3. Assign verified practitioners/care teams to permitted patients. Use role presets only as a readable starting point, with explicit effective privileges.

4. Extend My patients to direct and organization-assigned contexts. Require deliberate context selection; keep caches/queries separated by acting context.

5. On removal/suspension, invalidate effective organization access promptly; do not revoke the practitioner's independent direct connections.

### Gate: prove this before continuing

Staff cannot read another organization's patient or exceed category exclusions; patient sees organization connection; removal ends org access while valid direct access survives; admin without clinical privilege cannot finalize.

### Handoff and future dependency

T28 runs care under organization context; T29 limits assisted recovery to specially authorized staff. No broad role grants solely to make UI work.

TASK GROUP • EXECUTE SEQUENTIALLY

# T28 Implement organization registration and care operations

### Read before starting

Identity creation; scoped assignment; clinical-care services; organization design patterns.

### Create or change

Organization patient registration/search, care team worklists and narrow departmental request/result screens.

### Step-by-step

1. Register patients without email, phone or activated login using shared identity operations. Search controlled identifiers first; uncertain matches remain separate/provisional.

2. Record the organization’s own new care under an explicit authorized registration/care basis. This does not unlock another provider’s history without consent.

3. Expose assigned patient records, drafts, finalization and request/results through shared services with organization context and individual actor attribution.

4. Let explicitly privileged recipients submit/finalize requested results; keep departments as assignment/permission contexts, not independent unaccountable users.

5. Surface authorization pauses and review queues. Do not build hospital-wide operations, pharmacy fulfillment or specimen tracking.

### Gate: prove this before continuing

Unactivated patient receives attributed organizational care; cross-provider history stays unavailable without consent; assigned staff can perform only permitted work; every result identifies the signed-in contributor.

### Handoff and future dependency

T29 adds identity assistance screens. T30/T31 reconcile duplicate identities generated by real-world registration gaps without altering care provenance.

TASK GROUP • EXECUTE SEQUENTIALLY

# T29 Expose assisted activation and recovery safely

### Read before starting

Auth core T08; designated staff privileges; person lookup and evidence capture.

### Create or change

Organization assisted setup/reissue/recovery UI; support escalation queue and audit views.

### Step-by-step

1. Build a staff workflow to identify the patient, record verification method/evidence reference, and issue a private setup code or available email link.

2. For unactivated accounts, replacement atomically invalidates the old invitation. For activated accounts, use the separate recovery path and retire prior sessions.

3. Deliver credentials privately; never place secrets into permanent patient ID displays, audit logs or shared worklists. Patient sets the password in their own restricted session.

4. Escalate uncertain ownership to support without exposing history or allowing staff impersonation. Separate identity review evidence from ordinary clinical views.

5. Verify the patient path works without contact details and records subsequent contact additions independently.

### Gate: prove this before continuing

Authorized staff assists a no-contact patient end-to-end; ordinary staff cannot issue recovery; expired/replaced/replayed tokens fail; password is unknown to staff; old sessions cannot continue after recovery.

### Handoff and future dependency

T30 uses verified identity review patterns for duplicates, but possession of an invitation or ID alone is not sufficient merge ownership evidence.

TASK GROUP • EXECUTE SEQUENTIALLY

# T30 Detect and review duplicate identities

### Read before starting

Identity/alias data; registration provenance; designated merge reviewer privileges.

### Create or change

features/identity/duplicates/; review queue; match-evidence and conflict schemas.

### Step-by-step

1. Allow patient or ordinary authorized staff to flag a suspected duplicate without granting access to its medical history.

2. Generate limited candidates from permitted demographic/identifier data. Similar name, date or shared phone is a suggestion, never an automatic identity match.

3. Provide designated reviewers minimum necessary identity evidence, review reason and approval/rejection controls. Maintain provisional care while review is pending.

4. Resolve whether neither, one or both accounts are activated. Conflicting ownership or uncertain evidence escalates to Mhistree support.

5. Present the intended surviving identity rule and an access-preservation preview before executing any merge.

### Gate: prove this before continuing

Two records can remain safely separate while care continues; ordinary staff cannot approve; wrong-person candidate reveals no clinical history; ambiguous review cannot proceed to merge.

### Handoff and future dependency

T31 consumes only verified approvals and ownership evidence. Latest successful patient-login timestamps must be reliable and exclude staff activity.

TASK GROUP • EXECUTE SEQUENTIALLY

# T31 Merge reversibly without widening access

### Read before starting

Verified duplicate review; merge contract; origin-preserving grants; session invalidation; record kernel.

### Create or change

Identity merge/reversal operations; aliases; lineage journal; cache/index invalidation and tests.

### Step-by-step

1. Choose survivor: sole activated identity; otherwise oldest if neither; if both verified, most recent successful patient login. Escalate tied/missing evidence rather than guess.

2. Execute an atomic/concurrency-protected merge with an operation ID, immutable mapping journal, approval evidence and before/after grant lineage. Preserve original clinical source ownership.

3. Retire losing login and sessions/tokens. Keep prior IDs resolvable to survivor without treating aliases as access grants. Never copy permissions as an unrestricted union.

4. Update authorized views/indexes and invalidate affected caches. Pre-merge grants retain original provenance boundaries; new wider access requires fresh consent.

5. Implement controlled review/reversal with conflict handling for post-merge entries, consent and corrections. Restore person mapping only through audited rules; do not reactivate old credentials.

### Gate: prove this before continuing

Test all survivor cases, concurrent/replayed merge, rollback, no cross-hospital scope expansion, revoked grants, alias lookup, lost-session rejection and reversal after new records. No contribution or document disappears.

### Handoff and future dependency

T32 reruns cross-context safety after merges. If reversible mapping cannot be demonstrated, block merge completion and keep the duplicate-review workflow operational.

TASK GROUP • EXECUTE SEQUENTIALLY

# T32 Verify revocation, pauses and cross-context isolation

### Read before starting

All access consumers from feature map; patient/direct/org journeys; merge and AI contracts.

### Create or change

Cross-feature authorization suite; pause/resume integration tests; policy defects and fixes.

### Step-by-step

1. Enumerate reads, writes, downloads, AI retrieval, messages, appointments, draft finalization and result publication by connection and acting context.

2. Revoke during a live view, cached query, background job and in-flight AI response. Enforce current access at each sensitive delivery/mutation; remove stale UI state.

3. Verify preserved drafts/requests cannot be worked through the revoked path. Fresh consent and valid assignments restore eligibility; explicit practitioner review resumes work.

4. Test independent valid connections with explicit context choice. No automatic fallback or privilege union may bypass revocation or original provenance restrictions.

5. Include organization suspension, staff removal, patient category narrowing, merge/reversal and corrected-source invalidation. Fix failures at the shared policy owner.

### Gate: prove this before continuing

Every listed consumer has positive and negative evidence. No stale cache/download authorization creates new access after revocation. Document limits for copies already delivered; do not claim remote deletion.

### Handoff and future dependency

T33 hardens operation of the now-integrated system. Any failing security contract blocks release and its dependent acceptance rows.

TASK GROUP • EXECUTE SEQUENTIALLY

# T33 Add operational reliability and review controls

### Read before starting

Failure paths; job/audit metrics; export/closure queue; deployment and service constraints.

### Create or change

docs/operations.md; job dashboards/retry controls; backup/restore procedures; restricted support tooling.

### Step-by-step

1. Implement observability for processing failure, latency, queue age and authorization denial without exposing raw clinical content or secrets in routine logs.

2. Bound retries and dead-letter handling for extraction/notifications/index updates. Provide safe manual retry with idempotency and current permission checks.

3. Verify private storage and database backup/restore in a non-production environment. Track data/schema versions and recovery ownership.

4. Operationalize removal/closure/correction requests with visible status and auditable review. Do not promise a retention duration or deletion policy not yet validated.

5. Assign named review owners for clinical AI behavior, identity-verification procedures, data handling/retention and patient-facing explanations before launch. Record outcomes and unresolved blockers.

### Gate: prove this before continuing

A simulated parser outage recovers without data loss/duplicates; a backup restores a synthetic patient record and document; support roles cannot casually browse history; unresolved launch policy reviews remain visible.

### Handoff and future dependency

T34 uses verified recovery procedures for migration. Review records are release evidence, not a claim that this PDF supplies medical or legal approval.

TASK GROUP • EXECUTE SEQUENTIALLY

# T34 Migrate compatible data and rehearse cutover

### Read before starting

Audit reuse matrix; migration plan; new schemas; source provenance; backup/restore gate.

### Create or change

Versioned migration/import scripts; reconciliation report; cutover and rollback runbook.

### Step-by-step

1. Inventory actual old data separately from source code. Map patient IDs, account ownership, origins, contacts, records, documents, versions and permissions into target contracts.

2. Quarantine ambiguous ownership, unknown source status or unmappable grants for review; never infer identity merges or broaden consent to import successfully.

3. Run a dry migration on authorized non-production data. Compare counts, relationships, original files/checksums, aliases, timestamps and access expectations.

4. Make migration resumable and idempotent. Define maintenance window or a write-consistency strategy; test rollback before live cutover.

5. Prepare a concrete cutover plan and retain the previous source/data recovery path. Execute production-affecting work only under the developer’s deployment authorization.

### Gate: prove this before continuing

Reconciliation has no unexplained missing/duplicate clinical entries or grants; dry-run replay is safe; synthetic auth/session migration behavior is verified; rollback restores usable service.

### Handoff and future dependency

If there is no data to migrate, record verified evidence and mark import substeps not applicable. T35 still validates the new product independently of legacy tests.

TASK GROUP • EXECUTE SEQUENTIALLY

# T35 Run full MVP acceptance and release gate

### Read before starting

All task evidence; acceptance ledger; design baseline; migration/operations runbooks.

### Create or change

docs/release/acceptance.md; regression evidence; known-limitations and release readiness status.

### Step-by-step

1. Run the end-to-end scenarios in Appendix D against the integrated app, including real patient, direct practitioner and organization producer screens.

2. Verify auth and permissions under direct API calls, concurrency, retries and revoked context; test failures, not just happy-path clicks.

3. Compare representative mobile/desktop screens with preserved references; test keyboard access, focus, readable status and error recovery.

4. Confirm supported file types, service limits, AI failures, backups, data migration and required clinical/data-policy reviews have concrete evidence.

5. Classify every capability verified, blocked, deferred or not applicable. Mock-only behavior cannot satisfy a core MVP gate. Resolve blockers before declaring ready.

### Gate: prove this before continuing

All required core scenarios pass and release owners sign off their reviews; developer can reproduce evidence. No silent redesign, missing-contact requirement, unauthorized data path or undocumented incomplete integration remains.

### Handoff and future dependency

T36 packages the actual knowledge and operating instructions. Deployment is separate from declaring acceptance; record deployed version/environment only if it was actually deployed.

TASK GROUP • EXECUTE SEQUENTIALLY

# T36 Complete developer handover and portable memory

### Read before starting

AGENTS.md; WORKFLOW.md; feature map; all contract/readme/runbook links; release evidence.

### Create or change

Updated portable AGENTS.md; truthful WORKFLOW.md; README runbook; indexed task archive and design asset manifest.

### Step-by-step

1. Reconcile AGENTS.md with every accepted product change and verified design rule. Remove accidental audit-specific claims; keep it independently understandable in an empty folder.

2. Update WORKFLOW to the real finished/blocked state with exact file ownership, commands, test evidence, next actions and deferred features. Archive detailed logs while retaining an index.

3. Verify each feature README states public APIs, policy checks, schema ownership, tests, callers, future hooks and common failure locations.

4. Walk through a hypothetical patient-side bug using only the workflow map, patient README, relevant contract and direct imports; repair documentation that forces unnecessary whole-repository reading.

5. Provide clean setup, synthetic seed, test, migration, rollback and deployment instructions with environment names but no secrets. Include design assets or explicit asset dependencies for visual portability.

### Gate: prove this before continuing

An intermediate developer can trace and change one patient behavior without AI; a fresh-folder agent can reconstruct the product from AGENTS plus its declared design assets; no completed row lacks evidence; no unresolved blocker is hidden.

### Handoff and future dependency

Future changes start with the bounded maintenance procedure in Appendix C. Developer instructions remain the highest authority; these files must evolve with the product.

# Appendix A: AGENTS.md content contract

The implementing agent creates this file first. This is its required structure, not permission to leave headings empty. Populate the product sections from the canon in this playbook before reading legacy implementation details.

| Section | Required content |
|---|---|
| Mission and operating role | Build Mhistree as a careful product engineer/designer. Preserve UI; deliver a readable modular EHR. Never claim expertise substitutes for verification. |
| Authority and change control | Developer > two governing files >= playbook > legacy. Separate explicit decisions, recommended defaults and unresolved review gates; record revision dates and rationale. |
| Portable product specification | Identity/login/contact separation; activation/recovery; direct/org access; consent/revocation; merges; time/source/version semantics; AI boundaries; care workflows; first release and deferred scope. Copy substance, not just links. |
| Architecture and coding philosophy | Next.js feature-first ownership, route-only app/, DRY without premature abstraction, typed domain contracts, readable functions, server-only secrets, explicit error handling and minimal dependencies. |
| Design preservation | Initial rule to inspect; later verified tokens, layout and interaction rules, representative components and asset manifest. Do not substitute an invented style. |
| Execution and verification | Bounded context packet; prerequisite check; small task; positive/negative tests; truthful evidence; update workflow before proceeding. No unreviewed destructive migration. |
| Non-negotiable invariants | Identification does not authorize; no-contact patient supported; no permission expansion via merge; no automatic AI diagnosis; no loss of originals; no automatic paused-work restart. |
| Maintenance and reconstruction | How to use WORKFLOW and feature READMEs; how to revise rules with new developer instructions; dependencies needed for a clean-folder reconstruction. |

### Keep facts in their correct home

AGENTS holds enduring desired behavior and design rules. WORKFLOW holds “this module currently fails,” task status, branch paths and evidence. Feature READMEs hold local code maps. Avoid stale copies of the same status across all three.

### Portability acceptance

Copy AGENTS into an empty folder and read it without this PDF: the product, boundaries, build sequence and design principles must still be understandable. Exact branded imagery/fonts require the declared assets; a text file alone cannot recreate missing binary assets.

# Appendix B: WORKFLOW.md content contract

### Required top-level sections

Current objective and revision; repository/app roots; current commit/branch where available; baseline state; actual run/test commands; architecture/feature index; ordered task table; active task; blocked/deferred items; future-hook register; evidence index; recent handoffs. Keep long historical logs in linked task records.

### Required fields for every task

ID/title; status (not-started/in-progress/blocked/verified/deferred); goal; prerequisites; prior gate command/result; exact read-first files; intended files to create/change; schema/contract dependencies; ordered steps; expected outputs; positive/negative checks; known errors and handling; actual evidence; future consumers; next safe action.

### Task record example - structure only

T15 Persistent health details. Prerequisites: T09 atomic record kernel, T14 source-linked timeline. Read: AGENTS, UI_GUIDE, feature-map, clinical-records contract, health-records README, actual state/correction modules. Change: resolve exact paths during T05; never invent their existence. Verify: patient correction updates event-linked state, professional correction denied, concurrent revision conflict, failed transaction rollback. Future consumer: T16 retrieval, T22 practitioner corrections. Status: not-started until implementation evidence exists.

### Future-hook register

For each deferred integration record: contract name/version; current producer; future consumer task; files; fields/events; failure semantics; test fixture; readiness condition. Examples: organization context in AccessContext (T09 → T27), original provenance in contributions (T09 → T31), provider producers for patient appointment/messages (T18 → T25).

### End-of-task handoff format

Completed behavior; exact files touched; tests/commands and observed results; defects found/fixed; untested or blocked areas; schema/API changes and downstream consumers; design comparisons; next task and minimum read packet. Include date and commit if available. Never put credentials, recovery tokens or real patient records in documentation.

### Before/after discipline

Before: confirm previous work still passes its relevant smoke check and inspect current branch changes. After: update workflow, feature map and affected contracts; record any new decision in AGENTS only if it changes enduring rules. A checked checkbox without evidence is not verification.

### When a test cannot run

Name the exact missing dependency, command or credential category, its impact and the next action. Mark blocked or partially verified. Never replace a real integration requirement with a mock and call it complete.

# Appendix C: bounded maintenance and errors

## When the developer reports a patient-side bug

1. Read AGENTS and WORKFLOW current state. 2. Open the patient entry in feature-map and the owning feature README. 3. Reproduce the symptom and identify its route/action plus relevant contract. 4. Inspect that bounded file set and direct dependencies. 5. Fix at the responsible owner; add a regression check only where it proves the bug. 6. Verify affected consumers, update maps/evidence and report the behavior changed. Expand search only when evidence points outside the packet.

| Failure | Required handling |
| --- | --- |
| AUTH_REQUIRED / SESSION_RETIRED | Deny protected action; guide to login; do not return patient data in an error payload. |
| CONSENT_REQUIRED / ACCESS_REVOKED | Deny or pause; retain history; no silent fallback to another context. |
| IDENTITY_REVIEW_REQUIRED | Keep records separate/provisional; queue authorized review; avoid leaking possible-match history. |
| TOKEN_EXPIRED / TOKEN_USED | Do not redeem; offer authorized replacement path with non-enumerating response. |
| VERSION_CONFLICT | Preserve both source versions; prompt refetch/review; never silently overwrite. |
| EXTRACTION_PARTIAL / FAILED | Keep original and successful items, show gaps, safe retry; do not imply no relevant information. |
| PROVIDER_UNAVAILABLE / RATE_LIMITED | Retain user input; bounded retry and clear status; no fabricated AI response. |
| MIGRATION_RECONCILIATION_FAILED | Stop cutover; preserve source; report mismatch and use verified rollback. |

## Avoid overengineering

Prefer explicit operations such as finalizeClinicalEntry, revokeConnection or correctExtractedReport over universal “processEntity” handlers. Keep policies centralized but domain transitions specific. Explain non-obvious reasons in comments; let names and short functions explain ordinary mechanics.

## Handling a new developer decision

Record the new rule and affected task/contracts. Update AGENTS for enduring behavior and WORKFLOW for planned changes. Rerun only tests relevant to the changed risk plus required gates. Never quietly reclassify an accepted capability as deferred to make release easier.

# Appendix D: integrated acceptance scenarios

| Scenario | Observable proof |
|---|---|
| 1. No-contact patient | Organization registers person without contacts; care is recorded; patient later activates by one-time code, sets password, logs in with supported identifier and sees the same history. |
| 2. Natural health capture | Patient reports running yesterday and leg pain. Activity and symptom are linked, correctly timed and attributed. AI does not diagnose an injury or assume pain persists indefinitely. |
| 3. Partial document failure | Original survives extraction outage; readable facts keep document status; ambiguous units wait; retry does not duplicate; source preview remains private. |
| 4. Correction versus repeat | Patient corrects their extraction; cannot rewrite professional finding. Corrected result retains prior version; repeated test remains a distinct event. |
| 5. Direct care | Verified practitioner connects with consent, books appointment, texts, drafts/finalizes care, issues request and publishes result; patient sees permitted final content. |
| 6. Organization care | Approved organization assigns verified staff within consent; admin role alone cannot read/finalize; removal ends org access without ending valid direct access. |
| 7. Revocation mid-work | Draft/result/AI job loses authorization while running; no new unauthorized output; stored work preserved; fresh consent plus explicit review required to resume. |
| 8. Duplicate reconciliation | Two hospitals register same person separately. Verified merge keeps correct login, both histories and old-ID aliases; no wider grants; controlled reversal preserves later records. |
| 9. Recovery and isolation | Designated staff recover account without contacts; old sessions fail. Cross-patient IDs, private notes and unauthorized document downloads are denied. |
| 10. Operational failure | Retries, backups and restore verified; source and state remain consistent; live UI matches the preserved system; missing reviews/integrations block release honestly. |

Run with synthetic identities and documents. Each scenario records environment, command or manual steps, expected result, observed result, relevant files and evidence location. Automated success cannot stand in for clinical review or visual comparison.

# Sources, decisions and implementation notes

## Product sources and precedence

Primary: the developer’s supplied decisions and rebuild instructions through 08 October 2026, including the first-release reduction and approved defaults. Supporting context: Mhistree_Current_MVP_Context.md, read 08 October 2026 (101 lines). Later conversation decisions supersede its older guardianship, card/PIN and calendar scope. No source-code audit was performed for this PDF.

## Legacy PDF status

Mhistree_Complete_Architecture_and_Codebase_Specification.pdf was identified as a legacy artifact. Text extraction through the available read routes returned no readable content; no implementation claim from it was adopted. T02 requires the implementation agent to inspect applicable legacy evidence within the actual repository.

## External engineering references

The following official references informed the framework/auth boundary guidance only. The feature-first structure and task sequence are recommendations for Mhistree, not claims that a framework mandates them. Verify documentation against installed versions when executing the tasks.

[R1] Next.js - Project structure: https://nextjs.org/docs/app/getting-started/project-structure

[R2] Next.js - Authentication: https://nextjs.org/docs/app/guides/authentication

[R3] Next.js - Server and Client Components: https://nextjs.org/docs/app/getting-started/server-and-client-components

[R4] Supabase - Password-based Auth: https://supabase.com/docs/guides/auth/passwords

## Backend-specific execution note

If the implementation retains Supabase, read the applicable Supabase guidance and current docs before changing auth, policies or storage. Explicitly test row-level policies, server session freshness, privileged functions and private file access. The contactless login requirement remains a product invariant even if the auth adapter changes. No database, cloud account or production policy was modified while creating this document.

## Recommended defaults versus confirmed decisions

Confirmed product behavior is captured on the three canon pages and in the supplied task order. Root-level folder paths, feature names, error identifiers, internal schemas and module boundaries are implementation recommendations. The agent may refine them for clarity while preserving behavior, documenting the decision and keeping maps current.

## Definition of success

A working core Mhistree journey, a preserved interface, a readable modular codebase, verified access boundaries and truthful evolving documentation. Product work is not complete merely because screens render or an agent says it is done.